

Exact, Mergeable Time-Series Statistics for ClickHouse

Academic coursework submission

Continuation status: 16 September 2026

Abstract

This coursework studies how order-dependent time-series statistics can be implemented as ordinary ClickHouse aggregate functions without depending on row arrival, block boundaries, shard placement, or merge-tree shape. The implementation retains every finite (`timestamp`, `value`) pair, sorts by a unique timestamp, and merges canonical states by exact sorted union. This gives a direct associative and commutative merge contract on valid disjoint-key inputs, at the explicit cost of $O(n)$ persistent state.

The current source tree registers seven private-preview APIs: autocorrelation, the Ljung–Box test, Durbin–Watson, fixed-order lagged linear regression, a fixed-lag augmented Dickey–Fuller statistic, a KPSS statistic, and a one-mean-shift estimator. The four extensions deliberately return only quantities justified by their stated conventions: ADF and KPSS expose no p-values, while the change-point score is descriptive rather than calibrated. Regression uses a centered/scaled streaming Givens QR solver with explicit rank, conditioning, residual-resolution, and work guards. KPSS uses a function-local Bartlett bandwidth convention. Mean-shift finalization uses directly accumulated suffix moments to avoid cancellation.

Independent Python evidence, with seed 20260915, $n = 240$, and 120 repetitions, records coefficient recovery and directional diagnostic behavior. A separate Python-oracle benchmark measures only the batch reference algorithms and is not native ClickHouse performance evidence. Native evidence is recorded separately: Release and Debug focused GoogleTest both passed 38/38; SQL fixtures 05161–05164 passed 4/4 with no skips; the Distributed and `AggregatingMergeTree` paths passed. The only unresolved acceptance layer is remote CI: it is explicitly **BLOCKED**, not reported as a pass.

Contents

1	Research question and scope	2
2	State, merge law, and compact-state NO-GO decision	2
2.1	Why a compact boundary state is a NO-GO ordinary aggregate	3
3	Public API	3
4	Established diagnostics	4
5	Lagged linear regression	4
6	Fixed-lag ADF statistic	5
7	KPSS statistic	5

8	One-mean-shift estimator	6
9	Numerical policy shared by the family	7
10	Implementation and validation status	7
11	Extension experiment: independent Python evidence	8
12	Python-oracle benchmark, explicitly non-native	8
13	Limitations and threats to validity	9
14	Reproduction protocol	9
15	Conclusion	10

1 Research question and scope

ClickHouse reads and combines data in parallel. Rows may arrive from several parts and shards, and partial aggregate states may be joined in different parenthesizations. Physical arrival order therefore cannot stand in for chronological order [3, 4, 12]. The research question is: *which ordered time-series statistics can be exposed as ordinary mergeable aggregates, under exactly what state and numerical contracts?*

For one SQL group, a valid logical series is

$$X = ((t_0, x_0), \dots, (t_{n-1}, x_{n-1})), \quad t_0 < t_1 < \dots < t_{n-1},$$

where every value is finite and every key is unique. `timestamp` is an ordering key, not a duration. Lags count positions after sorting; unequal gaps, time zones, and calendar effects are ignored. A caller who needs equally spaced AR, ADF, or KPSS inference must resample first. Rows with a NULL argument are skipped by ClickHouse’s nullable combinator, which can itself change positional spacing.

The common `max_samples` parameter defaults to 1,000,000 and is limited to 10,000,000. Exceeding it is an error rather than truncation. The state is exact with respect to the retained canonical series but is not bounded-memory streaming and does not imply exact real arithmetic.

2 State, merge law, and compact-state NO-GO decision

The shared state stores all `(timestamp, Float64)` records. `add` appends in amortized $O(1)$ time and marks out-of-order input dirty. Before merge, serialization, or finalization, a dirty vector is sorted and duplicate keys are rejected. Two canonical vectors are joined by a two-pointer sorted union in $O(n_1 + n_2)$ work. The serialized extension envelope identifies the finalizer and its constant parameters; the delegated sample payload remains versioned and carries the sample cap.

For finite maps with disjoint keys, let $C(S)$ be their unique increasing-key representation and define

$$S \oplus T = C(S \cup T).$$

Because set union is associative and commutative and canonical sorting is unique,

$$(S \oplus T) \oplus U = S \oplus (T \oplus U), \quad S \oplus T = T \oplus S,$$

with the empty state as identity. Duplicate keys and cap overflow make the operation deliberately partial: any complete merge tree must eventually reject the same invalid logical union.

2.1 Why a compact boundary state is a NO-GO ordinary aggregate

The project ADR rejects a general $O(H)$ prefix/suffix envelope for ordinary ClickHouse aggregation. Consider singleton states with keys 1, 3, and 2. ClickHouse may merge 1 and 3 first. If that state forgets the interior and keeps only an envelope, it cannot later know that inserting key 2 must replace the apparent transition $1 \rightarrow 3$ by $1 \rightarrow 2$ and $2 \rightarrow 3$. For the lag-one product sum

$$T(X) = \sum_{i=1}^{n-1} x_{i-1}x_i,$$

the missing cross-boundary terms depend on information already discarded. Disjoint ranges are not necessarily adjacent, and keeping every disjoint subrange degenerates to $O(n)$ in the worst case.

A compact design would be valid only if a specialized execution operator guaranteed complete adjacent ranges, canonical left-to-right composition, and preservation of those conditions across spills, retries, remote aggregation, persisted states, and final coordinator merges. The ordinary aggregate interface provides none of these guarantees. Therefore no compact alternative is registered, exposed through `-State/-Merge`, or used to justify the seven APIs. This is a design decision, not an unfinished optimization.

3 Public API

All seven functions are private-preview parameterized aggregates over `(timestamp, value)`:

```
timeSeriesAutocorrelation(lag[, max_samples])(timestamp, value)
-> Float64
timeSeriesLjungBoxTest(max_lag[, model_df[, max_samples]])(timestamp, value)
-> Tuple(statistic Float64, p_value Float64)
timeSeriesDurbinWatson([max_samples])(timestamp, value)
-> Float64
timeSeriesLaggedLinearRegression(order[, max_samples])(timestamp, value)
-> Tuple(intercept Float64, coefficients Array(Float64))
timeSeriesADFStatistic(augmentation_lags[, deterministic[, max_samples]])(timestamp, value)
-> Tuple(statistic Float64, coefficient Float64, observations UInt64)
timeSeriesKPSSTest(regression[, bandwidth[, max_samples]])(timestamp, value)
-> Tuple(statistic Float64, bandwidth UInt64, observations UInt64)
timeSeriesMeanShiftChangePoint(min_segment[, max_samples])(timestamp, value)
-> Tuple(split_index UInt64, score Float64, mean_before Float64,
mean_after Float64, sse Float64)
```

Keys accept `UInt32`, `UInt64`, `DateTime`, or `DateTime64`. Values accept native integer and floating-point types and are converted to `Float64`; `Decimal` is not accepted. Duplicate timestamps and non-finite values are errors. The primary private-preview gate is `enable_time_series_aggregate_functions`; the factory also accepts the legacy compatibility gate `enable_time_series_table`.

4 Established diagnostics

Let $x_0, \dots, x_{n-1}$ be canonical values, with

$$\bar{x} = \frac{1}{n} \sum_{i=0}^{n-1} x_i, \quad M_2 = \sum_{i=0}^{n-1} (x_i - \bar{x})^2.$$

The biased, overall-mean autocorrelation at position lag h is

$$\rho_h = \frac{\sum_{i=h}^{n-1} (x_i - \bar{x})(x_{i-h} - \bar{x})}{M_2}.$$

Lag zero is 1 for a non-constant series. Empty, constant, or insufficient inputs produce `NaN`.

For $H = \text{max_lag}$, Ljung–Box is

$$Q(H) = n(n+2) \sum_{h=1}^H \frac{\rho_h^2}{n-h}, \quad p = \Pr\{\chi_{H-d}^2 \geq Q(H)\},$$

where $d = \text{model_df}$ and $0 \leq d < H$ [1,9]. This is the one API in the family that returns a calibrated tail probability, subject to its asymptotic assumptions.

Durbin–Watson is

$$DW = \frac{\sum_{i=1}^{n-1} (x_i - x_{i-1})^2}{\sum_{i=0}^{n-1} x_i^2}.$$

It is classically a residual diagnostic [6,7], but the aggregate fits no regression. Fewer than two samples or a zero denominator yields `NaN`.

5 Lagged linear regression

For fixed order p , the fitted positional autoregression follows the classical autoregressive lag construction [15]:

$$y_t = \alpha + \sum_{j=1}^p \phi_j y_{t-j} + \varepsilon_t, \quad t = p, \dots, n-1.$$

The returned coefficient array is ordered $(\phi_1, \dots, \phi_p)$. The order is fixed at aggregate creation, lies in [1,16], and must be below `max_samples`. Lagged design rows are constructed only after the full keyed state has been sorted. Locally fitted coefficients are never merged; merging local models would not equal fitting the global design.

The finalizer centers and scales response and predictor columns, then performs a streaming Givens QR factorization without column pivoting. It requires positive residual degrees of freedom, rejects rank-deficient or non-finite designs, and rejects a scaled reciprocal condition estimate below 10^{-12} . The checked work budget is

$$rp^2 \leq 100,000,000,$$

where $r = n - p$ is the number of design rows. Exceeding the budget or failing the numerical guards returns a fixed-shape result containing `NaN` values. Persistent state remains $O(n)$; QR finalization is $O(rp^2)$ with a small bounded matrix.

6 Fixed-lag ADF statistic

For augmentation lag p , the implemented regression is

$$\Delta y_t = d_t + \gamma y_{t-1} + \sum_{j=1}^p \psi_j \Delta y_{t-j} + \varepsilon_t, \quad t = p+1, \dots, n-1.$$

This is the fixed-lag augmented Dickey–Fuller regression [5, 11]. The deterministic mode is **none** (no deterministic term), **constant** (an intercept, and the default), or **trend** (an intercept and linear trend in canonical row position). The augmentation lag is fixed at aggregate creation and lies in $[0, 16]$. For a series long enough to form regression rows, the function returns γ , its regression t-ratio, and **observations** = $n - p - 1$. It performs no automatic lag selection and returns no p-value. The statistic is not a Student-t hypothesis test; ADF critical probabilities would require an audited response-surface convention not present in this API.

Sample admission follows the fixed-lag guard used by the official statsmodels **adfuller** implementation [13]:

$$p \leq \left\lfloor \frac{n}{2} \right\rfloor - d - 1,$$

where d is the number of deterministic terms (0, 1, or 2), followed by a positive residual-degrees-of-freedom check. The returned observation count remains informative even when the fit is undefined.

The same centered/scaled, non-pivoted Givens QR solver is used. Let $r = n - p - 1$ be the number of post-lag regression rows (and the returned **observations** count), and let c be the actual QR column count: $1 + p$ for **none** or **constant**, and $2 + p$ for **trend**; centering removes the explicit constant column but not its residual-degree-of-freedom cost. A fit is undefined if it is insufficient, singular, ill-conditioned ($\mathbf{rcond} < 10^{-12}$), non-finite, or if

$$rc^2 > 100,000,000.$$

An additional backward-error floor treats residual variance below Float64 resolution as unresolved instead of turning QR roundoff into an enormous t-ratio. Timestamp spacing is ignored: ADF interpretation requires the caller to supply defensibly equally spaced observations.

7 KPSS statistic

timeSeriesKPSSTest accepts **regression**=**'level'** or **regression**=**'trend'**. Let e_t be residuals after subtracting the sample mean in level mode or an intercept and canonical-position linear trend in trend mode. Define

$$S_t = \sum_{i=0}^t e_i, \quad \hat{\gamma}_h = \frac{1}{n} \sum_{t=h}^{n-1} e_t e_{t-h},$$

and the Bartlett/Newey–West long-run variance

$$\hat{\omega}_q^2 = \hat{\gamma}_0 + 2 \sum_{h=1}^q \left(1 - \frac{h}{q+1}\right) \hat{\gamma}_h.$$

The returned statistic is

$$KPSS = \frac{n^{-2} \sum_{t=0}^{n-1} S_t^2}{\hat{\omega}_q^2}$$

[8, 10]. The function returns the statistic, the resolved requested/default bandwidth parameter, and n ; that bandwidth field remains populated even when a guard makes the statistic undefined. It returns no p-value.

For $n \geq 2$, if bandwidth is omitted, this implementation uses its own explicit floor rule

$$q = \min \left(n - 1, \left\lfloor 12(n/100)^{1/4} \right\rfloor \right).$$

For $n < 2$, the resolved bandwidth is zero and the statistic is undefined. This convention must not be described as another library’s **legacy** mode. An explicit q is non-negative and capped at 1024; creation requires $q < \text{max_samples}$, while a defined statistic additionally requires $q < n$. Direct finalization is $O(nq)$ and returns an undefined statistic when $nq > 100,000,000$, the long-run variance is non-positive/non-finite, or the detrended series has no resolvable variation. The equal-spacing caveat applies here as strongly as for ADF.

8 One-mean-shift estimator

For every legal split k satisfying $\text{min_segment} \leq k \leq n - \text{min_segment}$, define

$$SSE(k) = \sum_{i < k} (x_i - \bar{x}_{0:k})^2 + \sum_{i \geq k} (x_i - \bar{x}_{k:n})^2.$$

The chosen split minimizes this objective. If

$$SSE_0 = \sum_{i=0}^{n-1} (x_i - \bar{x})^2,$$

the descriptive score is

$$\text{score} = \max \left(0, 1 - \frac{SSE(k^*)}{SSE_0} \right).$$

It is the fraction of one-mean variation removed by a two-mean fit, not a p-value and not a general multiple-change procedure. **min_segment** must be positive and cannot exceed **max_samples**/2. **split_index** is the number of samples in the left segment. No identifiable improvement returns **split_index**=0 with NaN fields.

The native finalizer range-scales values, maintains a running prefix Welford moment [2, 14], and stores directly accumulated suffix Welford moments. Reconstructing suffix SSE by subtraction from total and prefix terms was rejected because a strong break can make that subtraction catastrophically cancel. The native scan is $O(n)$ time with $O(n)$ transient suffix memory in addition to the $O(n)$ persistent keyed state. The independent Python oracle intentionally uses a direct $O(n^2)$ slice scan.

Let $\gamma_n = n\varepsilon/(1-n\varepsilon)$, where ε is binary64 machine epsilon. The finalizer accepts a later candidate only when incumbent – candidate $> 8\gamma_n \max(|\text{candidate}|, |\text{incumbent}|)$; this count-aware envelope prevents accumulated Welford rounding on a long no-improvement series from manufacturing a change, and otherwise retains the earliest canonical split. After rescaling, a positive SSE too large for **Float64** is returned as **+Inf** without discarding an otherwise valid split; an extremely small SSE may underflow to zero. The dimensionless score can remain valid in both cases.

9 Numerical policy shared by the family

ACF and Ljung–Box use a midpoint/range coordinate transform; Durbin–Watson divides by the maximum absolute value. Compensated sums reduce avoidable cancellation. Regression columns are centered and scaled before QR. KPSS and mean-shift calculations similarly operate in scaled coordinates. These transformations preserve the stated dimensionless quantities in exact arithmetic, but cannot recover information already lost when an input is converted to `Float64`.

Undefined outcomes are part of the contract, not silent success. Depending on the API, they appear as `NaN` scalar or tuple fields while counts or bandwidth may remain populated. The independent Python oracle checks ordinary numerical formulas; it is not a bitwise or edge-case API oracle: it may raise `ValueError`, uses a strict change-point comparison, and omits native work guards. Cross-platform validation must use absolute and relative tolerances and separately check finiteness, `NaN`, bounds, and deterministic tie behavior.

10 Implementation and validation status

The checkout on branch `coursework/time-series-extensions` contains the original diagnostics state and wrapper plus `AggregateFunctionTimeSeriesStatisticalExtensions.h/.cpp`. The extension state delegates keyed storage and canonical merging to the same exact sample state, while its envelope records finalizer kind and constant parameters. The global registry calls both registration functions. Three additional SQL/reference fixtures (05162–05164, four fixtures total with 05161) and a focused extension GoogleTest source are present in the working tree.

Source presence and registration are not equivalent to validated native execution. The following ledger records completed local native evidence separately from the still-unavailable remote CI layer:

Native acceptance layer	Current status
Release configure/build	PASS : full Release build completed 6838/6838; current revision <code>1ad279671de9cdda088fb64046d6ae1d4e7f854f</code> , binary SHA-256 <code>c0753569f7b2c1abc1c41e3e5ae57c5834f4094bff879df64dac236eba33eac4</code> .
Seven-API focused GoogleTest	PASS : 38/38 in Release acceptance and again 38/38 in the focused Debug run; packages <code>evidence/native-acceptance-20260916-58b61c3a/</code> and <code>evidence/debug-gtest-20260916-1ad279671/</code> .
SQL fixtures 05161–05164	PASS : 4/4, zero skipped and zero failed; logs are in <code>evidence/native-acceptance-20260916-58b61c3a/</code> .
Distributed and Aggregating-MergeTree	PASS : both paths, including duplicate-key error propagation, are included in the native acceptance package.
Required remote CI	BLOCKED : GitHub Actions on the fork exposes no usable workflow/runners to this checkout. Local results are not CI results.
Native Release benchmark	PASS : 92 measured rows in <code>evidence/native-benchmark-20260916-58b61c3a/</code> ; state/merge evidence contains 123 direct, 192 state-size, and 96 merge rows in <code>evidence/state-merge-benchmark-20260916-1ad279671/</code> .

The earlier three-diagnostic revision is not used as proof for the extensions. The stated Release/Debug runs cover all seven APIs; SQL acceptance covers dispatch and serialization-oriented paths, and the Distributed/**AggregatingMergeTree** cases cover merge execution. Each evidence package contains a SHA-256 manifest verified after the run. Remote CI remains a distinct blocked gate.

11 Extension experiment: independent Python evidence

The extension experiment uses seed 20260915, $n = 240$, 120 repetitions, fixed ADF lag 1, and `min_segment=20`. The final provenance-complete LF-normalized run took 4.170579 seconds under Python 3.12.6 on Windows. The recorded tables contain 360 AR-fit rows, 240 ADF rows, 480 KPSS rows, 120 mean-shift rows, 12 edge outcomes, and one optional-library cross-check row. Calculations use the independent coursework batch oracle; optional statsmodels was available only as a cross-check. These are statistical/reference results, not execution of the C++ aggregates.

Experiment	Verified result
AR(1), $\phi = 0.25$, noise SD 0.2	120/120 fits; intercept bias 0.002854 and RMSE 0.034467; ϕ_1 bias -0.007942 and RMSE 0.063852.
AR(1), $\phi = 0.70$, noise SD 1.0	120/120 fits; intercept bias 0.009558 and RMSE 0.089170; ϕ_1 bias -0.007259 and RMSE 0.051025.
AR(2), $\phi = (0.50, -0.25)$	120/120 fits; intercept bias 0.004495 and RMSE 0.049320; ϕ_1 bias -0.005070 and RMSE 0.062453; ϕ_2 bias -0.007596 and RMSE 0.061536.
ADF direction	Stationary mean -7.76810 ; random-walk mean -1.56964 ; stationary more negative in 120/120 pairs.
KPSS level behavior	Level-stationary mean 0.16800; random-walk mean 0.86441; level smaller in 113/120 pairs (0.9417).
KPSS trend behavior	Detrended mean 0.07503; level-only trend mean 1.70315; detrended smaller in 120/120 pairs.
Mean-shift localization	MAE 0.1083 samples; exact in 109/120 (0.9083); within 12 samples in 120/120.

ADF and KPSS rows are directional comparisons only. Because the APIs return no p-values, they make no calibrated rejection-rate claim. Optional statsmodels cross-check differences were approximately 5.33×10^{-15} for ADF, zero for the AR intercept, 1.67×10^{-16} and 3.05×10^{-16} for the two AR coefficients, and 1.39×10^{-17} for KPSS. They support agreement of this fixture, not universal equivalence across all inputs and conventions.

The recorded edge grid contains constant, short, and NULL-filtered cases. It checks explicit undefined outcomes, usable values where defined, and the constant-series mapping to `split_index=0`. It complements, rather than replaces, the completed C++ and SQL runs.

12 Python-oracle benchmark, explicitly non-native

The extension benchmark times independent batch-oracle finalizers on Windows/Python 3.12.6. Input generation and one warm-up are outside each timed sample; each case has three timed repetitions. `tracemalloc` measures Python-traced allocation, not process RSS and not a ClickHouse allocator. The change-point oracle is intentionally $O(n^2)$, unlike the native $O(n)$ finalizer.

Python oracle finalizer	Configuration at $n = 4096$	Median ms	Peak bytes
Lagged regression	$p = 1$	167.348	716,160
Lagged regression	$p = 8$	305.731	850,248
ADF	$p = 0$, constant	214.918	847,840
ADF	$p = 4$, constant	380.251	981,260
KPSS	trend, $q = 0$	62.994	625,260
KPSS	trend, $q = 32$	419.694	625,260
Mean shift	<code>min_segment=8</code>	18,541.648	359,088

The benchmark spans $n \in \{256, 1024, 4096\}$, AR orders $\{1, 4, 8\}$, ADF lags $\{0, 2, 4\}$, and KPSS bandwidths $\{0, 8, 32\}$. Its only defensible interpretation is algorithmic behavior of the independent Python oracle on one host. It provides no ClickHouse throughput, query-plan, vectorization, RSS, serialization-size, or Release-build claim.

13 Limitations and threats to validity

- Every function retains all accepted samples. `max_samples` makes exhaustion explicit but does not make the state suitable for unlimited histories.
- Dirty sorting costs $O(n \log n)$; Ljung–Box costs $O(nH)$; regression and ADF cost $O(rc^2)$; KPSS costs $O(nq)$; native mean shift uses $O(n)$ transient memory.
- Lags and trends are positional. Unequal timestamps are not repaired, and skipped NULL rows alter the position sequence.
- ADF has fixed caller-selected lag and no p-value or MacKinnon calibration. KPSS has a local bandwidth rule and no p-value. Their statistics alone do not prove stationarity or nonstationarity.
- Lagged regression supplies no forecast intervals or automatic order selection. Mean shift assumes at most one mean change and supplies no false-positive calibration.
- Non-pivoted QR and fixed `rcond` intentionally reject some difficult but identifiable designs. The resolution policy may classify genuine noise below the Float64 floor as unresolved.
- The Python experiment uses one length, one seed, and 120 repetitions. Its frequencies are Monte Carlo observations, not theoretical probabilities.
- The Python benchmark is not native. Native Release build, focused gtest, SQL/Distributed execution, and native performance have been recorded locally, but remote CI remains **BLOCKED** and cross-platform performance/allocation coverage is not exhaustive.
- The branch is coursework work in a fork; registration metadata is not evidence that these functions have entered an official ClickHouse release.

14 Reproduction protocol

From the manuscript directory, the independent extension evidence can be reproduced with:

```

py -3 -m unittest discover -s ../reference/python -p "test_*.py"
py -3 ../evidence/experiments/run_extension_experiments.py \
  --seed 20260915 --n 240 --reps 120 --adf-lags 1 \
  --change-min-segment 20 --output-dir <new-output-directory>
py -3 ../evidence/benchmarks/benchmark_extensions.py \
  --output-dir <new-benchmark-directory> --seed 20260915 \
  --n 256,1024,4096 --ar-orders 1,4,8 --adf-orders 0,2,4 \
  --kpss-bandwidths 0,8,32 --change-point-n 256,1024,4096 \
  --min-segment 8 --warmup 1 --repetitions 3

```

Checked-in experiment evidence is under `evidence/experiments/extension_results_20260915_final_v4_trusted/`; the fresh oracle benchmark is `evidence/benchmarks/extensions-20260916-final/`. Native acceptance is `evidence/native-acceptance-20260916-58b61c3a/`, native performance is `evidence/native-benchmark-20260916-58b61c3a/`, state/merge performance is `evidence/state-merge-benchmark-20260916-1ad279671/`, and focused Debug evidence is `evidence/debug-gtest-20260916-1ad279671/`. The seven generated documentation pages pass their exact generator checks, and the isolated documentation runner reports 7/7 selected examples successful in `evidence/docs-examples-20260916-1ad279671/`. Python checks also passed: 25/25 trusted reference tests and 6/6 experiment tests. Every listed package carries a verified SHA-256 manifest. Remote CI is the remaining blocked gate.

15 Conclusion

The central result is architectural. Exact order-dependent statistics can behave as ordinary distributed aggregates when state retains the complete keyed sample and merge is canonical sorted union. The 1,3,2 counterexample shows why a compact prefix/suffix envelope is not closed under arbitrary ClickHouse merge trees.

Seven private-preview APIs now exist in source with explicit formulas and resource guards. The extensions narrow their claims deliberately: fixed-order AR coefficients, a fixed-lag ADF t-statistic without a p-value, KPSS under a stated Bartlett bandwidth convention without a p-value, and a descriptive one-break mean-shift objective. Independent Python experiments and benchmarks make the mathematics inspectable; completed local native Release/Debug/gtest/SQL/Distributed/benchmark evidence makes the implementation auditable. Remote CI is deliberately kept separate as **BLOCKED**, rather than being inferred from local success.

References

- [1] George E. P. Box and David A. Pierce. Distribution of residual autocorrelations in autoregressive-integrated moving average time series models. *Journal of the American Statistical Association*, 65(332):1509–1526, 1970.
- [2] Tony F. Chan, Gene H. Golub, and Randall J. LeVeque. Algorithms for computing the sample variance: Analysis and recommendations. *The American Statistician*, 37(3):242–247, 1983.
- [3] ClickHouse. Data parts, 2026.
- [4] ClickHouse. MergeTree table engine, 2026.

- [5] David A. Dickey and Wayne A. Fuller. Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366a):427–431, 1979.
- [6] James Durbin and Geoffrey S. Watson. Testing for serial correlation in least squares regression. i. *Biometrika*, 37(3–4):409–428, 1950.
- [7] James Durbin and Geoffrey S. Watson. Testing for serial correlation in least squares regression. iii. *Biometrika*, 58(1):1–19, 1971.
- [8] Denis Kwiatkowski, Peter C. B. Phillips, Peter Schmidt, and Yongcheol Shin. Testing the null hypothesis of stationarity against the alternative of a unit root. *Journal of Econometrics*, 54(1–3):159–178, 1992.
- [9] Greta M. Ljung and George E. P. Box. On a measure of lack of fit in time series models. *Biometrika*, 65(2):297–303, 1978.
- [10] Whitney K. Newey and Kenneth D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987.
- [11] Said E. Said and David A. Dickey. Testing for unit roots in autoregressive-moving average models of unknown order. *Biometrika*, 71(3):599–607, 1984.
- [12] Sven Schulze et al. ClickHouse: Lightning fast analytics for everyone. *Proceedings of the VLDB Endowment*, 17(12):3731–3744, 2024.
- [13] statsmodels developers. Source code for statsmodels.tsa.stattools._stattools, 2026.
- [14] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.
- [15] G. Udny Yule. On a method of investigating periodicities in disturbed series, with special reference to wolfer’s sunspot numbers. *Philosophical Transactions of the Royal Society A*, 226:267–298, 1927.